

第1章 绪论

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 复习全书的地基数学:指数、对数、级数、模运算,并知道它们在算法中的用途
- 理解"程序 = 数据结构 + 算法",建立正确性与效率两个评判维度,预告复杂度记号
- 掌握**递归的四法则**,能写出阶乘等经典递归并分析其递推式
- 完成 C 到 C++ 的第一波迁移:cin/cout、string、vector、class,能独立编译运行第一个程序
- 亲手实现斐波那契的朴素递归与记忆化,体会 $O(2^n)$ 与 $O(n)$ 之别
- 理解调用栈与栈帧,知道深层递归为何段错误、如何规避;学完能独立实现汉诺塔与全排列

💡 从 C 到 C++:本章主题如何迁移

先给全书的一句话直觉:数据结构决定"数据怎么放",算法决定"问题怎么做",两者合起来决定"程序跑多快"。第1章是唯一的"地基章",地基不牢,后面 11 章都要还债。

语言迁移上,你已有的 C 技能全部有效——C++ 是 C 的超集。本章只补三样最常用的新工具:cin/cout 输入输出、string 与 vector (自动管理内存的动态数组)、class (把数据和操作打包),它们覆盖本书 90% 的示例代码。

算法直觉上,本章最重要的思想是**递归**:把大问题缩小成"同样结构的小问题"。同一个斐波那契,笨写是指数级,加一行"记忆化"就变成线性——这是全书第一次"复杂度决定生死"的演示。

📦 前置知识自查

数学前置:高中数学:幂运算与对数(§1.1 会系统复习);求和记号 Σ 会读即可。

C 语言前置:函数定义与调用、for/while 循环、数组与下标、printf/scanf、struct 与指针(会更好)。

依赖的前面章节:无,本章是全书第一章。

🚀 本章学习路线

1. **第一阶段(1 小时):**读 §1.1–§1.2,动手验证取模等例子,背里程碑数值
2. **第二阶段(1.5 小时):**读 §1.3–§1.4,编译运行 §1.4 完整小程序并改造
3. **第三阶段(1.5 小时):**读 §1.5–§1.6,运行两版斐波那契,看 n 从 30 到 40 时间如何爆炸
4. **第四阶段(实验,1 小时):**完成章末实验:汉诺塔与全排列,验证 $2^n - 1$ 、 $n!$
5. **验收标准:**能默写递归四法则;能独立写出两版斐波那契;能解释深层递归为何崩溃

本章知识导图

- §1.1 数学复习:指数规则、对数与换底、级数三公式、模运算与负数取模(C++/Python 差异)
- §1.2 算法概念:程序 = 数据结构 + 算法、正确性与效率、复杂度记号预告
- §1.3 递归简介:递归四法则、C 递归与调用栈回顾、阶乘示例
- §1.4 从 C 到 C++ 速成:输入输出、string/vector、函数、类入门、编译与完整小程序
- §1.5 完整示例:斐波那契——朴素递归(指数级)vs 记忆化(线性)、手写 vs 标准库
- §1.6 递归的工程注意:调用栈与栈帧、栈溢出与段错误、如何避免深层递归
- §1.7 全书路线图:12 章一句话导航

§1.1 数学复习

1.1.1 指数规则

指数是全书最频繁出现的运算,下列规则必须刻进直觉:

- $a^0 = 1 (a \neq 0); a^1 = a; a^{-n} = 1/a^n$
- **同底数相乘,指数相加:** $a^m \cdot a^n = a^{m+n}$
- **幂的幂,指数相乘:** $(a^m)^n = a^{m \cdot n}$
- **同底数相除,指数相减:** $a^m / a^n = a^{m-n}$

背几个里程碑数值: $2^{10} = 1024; 2^{20} \approx 10^6; 2^{30} \approx 10^9$ 。反过来: $10^6 \approx 2^{20}$,对 100 万数据"每次减半"地查找最多约 20 步——第2章对数复杂度的直觉起点。

记号说明(全书约定,首次出现)

本书数学记号统一:幂用上标(n^2 、 2^n);对数底数用下标($\log_2 n$);递推式行内用等宽代码($T(n) = T(n-1) + O(1)$);求和用 $\Sigma(\Sigma_{i=1}^n i = n(n+1)/2)$;模运算正文写 $a \bmod b$,代码里一律写 $a \% b$ 。

1.1.2 对数

对数的定义: $\log_b X = a$ 当且仅当 $b^a = X (b > 0, b \neq 1, X > 0)$ 。直觉:对数回答"几次方"——"2 的几次方等于 8"?答 $\log_2 8 = 3$ 。对数与指数互为逆运算,是"减半""倍增"类算法(二分查找、快速幂)的天然度量。

换底公式: $\log_b X = \log_a X / \log_a b$ 。推导:设 $\log_b X = c$,则 $b^c = X$,两边对 a 取对数即得 $c \cdot \log_a b = \log_a X$ 。工程含义:换底只差常数倍,而常数倍会被大 O 记号吞掉(第2章),所以**复杂度分析中底数不重要**。

记法约定:计算机领域 $\log$ 默认以 2 为底(算法世界是"减半"与"倍增"的世界); $\ln$ 以 $e \approx 2.718$ 为底; $\lg$ 有时指以 10 为底。里程碑: $\log_2 1024 = 10; \log_2 10^6 \approx 20$ 。对数"几乎不增长":规模翻 1000 倍, $\log_2 n$ 只从 10 长到 20。

1.1.3 级数

三个级数公式是第2章复杂度计算的原料,先记住结论与量级:

- **算术级数:** $\Sigma_{i=1}^n i = n(n+1)/2 \approx n^2/2$ ——双重循环的总迭代次数,量级 $O(n^2)$ 。例: $1+2+\dots+100 = 5050$

- **几何级数**: $\sum_{i=0}^n 2^i = 2^{n+1} - 1$ ——倍增翻倍的总量,量级 $O(2^n)$ 。例: $1+2+4+8+16+32 = 63$;推广: $\sum_{i=0}^n r^i = (r^{n+1} - 1)/(r-1)$
- **调和级数**: $\sum_{i=1}^n 1/i \approx \ln n$ ——量级 $O(\log n)$,出现在快速排序平均分析的求和中

记法: $\sum_{i=1}^n f(i)$ 表示把 i 从 1 到 n 逐一代入 $f(i)$ 再求和。第2章会把循环迭代次数翻译成这样的求和——两个记号是一回事。

1.1.4 模运算

模运算: $a \bmod b$ 表示 a 除以 b 的余数,数学上写成 $a = q \cdot b + r$;但"余数"取决于"商怎么取整",这是各语言分歧的根源(★小节)。两条最重要的性质(散列、循环数组、快速幂的基础):

- $(a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
- $(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$

★ 重点:C/C++ 与 Python 的负数取模不一样

关键分歧在**负数**:C/C++ 除法"向 0 截断"(truncation),余数符号与被除数相同;Python 除法"向下取整"(floor),余数符号与除数相同。同一表达式两种语言答案不同:

C/C++ 的写法(商向 0 截断)	Python 的写法(商向负无穷取整)
<code>-7 % 3</code> 结果是 -1 (商截断为 -2,余 $-7 - (-2) \cdot 3 = -1$)	<code>-7 % 3</code> 结果是 2 (商取整为 -3,余 $-7 - (-3) \cdot 3 = 2$)
<code>7 % -3</code> 结果是 1 (商 -2,余 1)	<code>7 % -3</code> 结果是 -2 (商 -3,余 -2)
<code>-7 / 3</code> 结果是 -2 (截断)	<code>-7 // 3</code> 结果是 -3 (向下取整)

两者"同号"时行为一致,只在**异号**时分歧;判奇偶、判整除不受影响。本书代码都在非负语境用 `%`,但移植代码时它会悄悄改变结果,务必知道这个坑。

⚠ 误区 1:默认负数取模结果是正数

C++(与 C 相同)中 `-7 % 3` 是 **-1**:余数符号随被除数。要"数学意义的非负余数",用惯用法: `int r = a % b; if (r < 0) r += b;`。同源陷阱是除法截断: `-7 / 3 == -2`,与"向下取整"的直觉相反,二分查找中写错就是越界。

⚠ 误区 2:int 溢出是未定义行为

C++ 的 `int` 是 32 位,范围约 $\pm 2.1 \times 10^9$; `INT_MAX + 1` 回绕成负数是**未定义行为**——可能负数、可能崩溃、可能被优化器"整段删除"。斐波那契 $F(47)$ 就超过 `int` 范围(\$1.5)。本书规则:可能变大的整数一律用 `long long`。

示例 1-1 C++ 负数取模实测

```
// C 与 C++ 相同:取模是"向 0 截断"除法的余数
// 公式:余数 = 被除数 - 商 × 除数,商向 0 取整
#include <iostream>
using namespace std;
int main() {
    cout << -7 % 3 << endl;    // -1:商取 -2,余数 -7 - (-2)*3 = -1
    cout << 7 % -3 << endl;    // 1:商取 -2,余数 7 - (-2)*(-3) = 1
    cout << -7 / 3 << endl;    // -2:除法同样向 0 截断,不是向下取整!
    int r = -7 % 3;            // 惯用法:修正成非负余数
    if (r < 0) r += 3;
    cout << r << endl;        // 2:与 Python 的 -7 % 3 一致了
    return 0;
}
```

§1.2 算法概念

1.2.1 程序 = 数据结构 + 算法

瑞士计算机科学家 Niklaus Wirth 的名言: **程序 = 数据结构 + 算法**。**数据结构**回答"数据如何组织",**算法**回答"问题如何求解"。经典例子——电话簿:按姓名排序可以二分查找(第2章)瞬间定位;打乱顺序只能逐个翻。数据没变,变的是"组织方式 + 查找算法"。

1.2.2 正确性与效率

一个解法要配得上"算法"二字,必须同时满足:**正确性**——对一切合法输入给出正确输出;**效率**——时间与空间可接受。两条缺一不可:正确但慢到跑不完,等于不能用;快但结果错,比慢更危险。

例:求 2^n ,连乘 n 次要一百万次乘法($n = 10^6$);第2章的快速幂约 20 次。本书尺度:先保证正确,再追求高效;复杂度分析(第2章)给"效率"机器无关的精确度量,本章只需建立直觉:数据规模变大时,运行时间如何增长。

1.2.3 复杂度记号预告

全书用大 O 记号描述效率。先混个脸熟,第2章给严格定义:

- $O(1)$ 常数:步数与规模无关——按下标取元素
- $O(\log n)$ 对数:规模翻倍只多一步——二分查找
- $O(n)$ 线性:遍历一趟——线性查找、求和
- $O(n \log n)$ 线性对数:归并排序、快排(平均)
- $O(n^2)$ 平方:双重循环——插入排序
- $O(2^n)$ 指数:穷举、朴素递归斐波那契——规模稍大就不可用

增长阶排序(背诵):常数 < 对数 < 线性 < 线性对数 < 平方 < 立方 < 指数。本章递归将用到两个结论: $T(n) = T(n-1) + O(1)$ 的解是 $O(n)$ (阶乘), $T(n) = T(n-1) + T(n-2) + O(1)$ 的解是 $O(2^n)$ (朴素斐波那契)。

§1.3 递归简介

1.3.1 递归四法则

Weiss 教材把递归写成四条法则,每条都配一个"违反后果":

1. **基准情形(base case)**:必须存在不需要递归、直接得出答案的情形。没有基准情形 = 无限递归。
2. **不断推进**:每次递归调用都必须让问题朝基准情形靠近。不推进 = 死循环;推进过快跳过基准情形 = 边界错误。
3. **设计法则**:假设所有递归调用都正确,只设计"当前层如何利用子问题的答案"。不要手工跟踪每一层——人脑栈只有几层深,程序栈可以几十万层。
4. **合成效益法则**:不要在递归调用的不同分支中重复求解同一个问题实例。重复求解会让复杂度爆炸成指数级——§1.5 的朴素斐波那契正是反面教材。

1.3.2 C 递归与调用栈回顾

你学 C 时写过递归,现在补上底层机制。程序运行时,系统维护一块**调用栈**(call stack):每次函数调用,系统把"返回地址 + 参数 + 局部变量"打包成**栈帧**(stack frame)压入;函数返回时弹出。递归没有任何特殊——递归调用也是普通调用,只是被调用的函数与自己同名。

由此得两个推论:① **递归深度 = 同时存活(尚未返回)的栈帧数**;② 递归每层多一个栈帧,这是迭代没有的成本。层数过多时栈空间耗尽——进程直接崩溃(§1.6 详解)。C 与 C++ 的递归写法逐字符一致。

1.3.3 阶乘示例

阶乘 $n! = n \times (n-1) \times \dots \times 1$,基准情形 $0! = 1$,递归定义 $n! = n \cdot (n-1)!$,递推式 $T(n) = T(n-1) + O(1)$,解为 $O(n)$ 。

示例 1-2 阶乘:递归版与迭代版

```
// 递归版:把 n! 分解为 n * (n-1)!, n <= 1 是基准情形
// 与 C 的写法完全一致——递归是语言无关的思想
long long factRec(int n) {
    if (n <= 1) return 1;           // 基准情形:0! = 1! = 1
    return n * factRec(n - 1);      // 不断推进:n 越来越小
}

// 迭代版:循环乘过去,不占调用栈
long long factIter(int n) {
    long long r = 1;
    for (int i = 2; i <= n; ++i)
        r *= i;
    return r;
}

// 验证:factRec(10) = factIter(10) = 3628800
```

§1.4 从 C 到 C++ 速成

1.4.1 输入输出:printf/scanf → cin/cout

C++ 的 `iostream` 用 `cin` (读入)与 `cout` (输出)替代 `scanf/printf`: `cout << x` 输出 `x`, `cin >> x` 读入 `x`,可连续拼接 (`cin >> a >> b`),自动按类型解析,**不需要 %d/%f/%s 格式符**——格式符写错(如 %d 配 long long)是 C 程序最常见的崩溃源之一。

C 的写法	C++ 的写法
<code>printf("n = %d\n", n);</code>	<code>cout << "n = " << n << endl;</code>
<code>scanf("%d", &n);</code>	<code>cin >> n;</code>
<code>scanf("%s %d", name, &score);</code>	<code>cin >> name >> score;</code>
格式符写错 = 静默读错/崩溃	类型由编译器推导,无格式符

注意: `cin >>` 自动跳过空白;读整行用 `getline(cin, s)`;混用时先 `cin.ignore()` 吃掉残留回车——C++ 入门第一坑。

1.4.2 字符串与动态数组:char*/malloc → string/vector

C 里动态数组要自己管"长度、容量、扩容、释放"四件事; `std::string` 与 `std::vector` 全部接管:自动扩容、离开作用域自动释放(RAII,第3章细讲)、自带长度 `size()`。学习阶段理解底层,工程阶段直接用标准库。

C 的写法	C++ 的写法
<code>int *a = malloc(n * sizeof(int));</code>	<code>vector<int> a(n);</code>
<code>a[len++] = x;</code> (自己管长度)	<code>a.push_back(x);</code> (自动扩容)
<code>strcpy(buf, s); strcat(buf, t);</code>	<code>string s = "hello"; s += " world";</code>
<code>free(a);</code> (忘记即泄漏)	离开作用域自动释放

示例 1-3 string 与 vector 速览

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
int main() {
    string s = "hello";           // 赋值即拷贝,无需 strcpy
    s += " world";               // 拼接,无需 strcat
    cout << s.size() << " " << s[0] << endl;    // 11 h
    vector<int> v = {3, 1, 4};    // 初始化列表,无需手动 malloc
    v.push_back(1);              // 尾部插入,自动扩容
    v[0] = 7;                    // 按下标访问,0(1)
    cout << v.size() << " " << v[0] << endl;    // 4 7
    for (int x : v)               // 范围 for:遍历容器
        cout << x << " ";
    return 0;
}
```

1.4.3 函数:C 的函数在 C++ 中全部有效

C++ 函数是 C 函数的超集:返回值、参数、递归——写法完全不变。新增三样:引用传参 `void f(int& x)` ——直接改实参,替代 C 的指针传参;函数重载——同名函数按参数类型区分;类型常由编译器推导(`auto`)。范围 `for`(`for (int x : v)`)是遍历容器的标准姿势,不会越界。

1.4.4 类入门:struct+函数 → class

C 里"数据 + 操作"分离:struct 存数据,全局函数操作它,每次调用传指针。C++ 的 `class` 把数据与操作封装成一个整体, `obj.func()` 自动绑定到该对象。这是全书所有数据结构(栈、树、堆.....)的呈现形式,第3章从抽象数据类型(ADT)角度深化。

示例 1-4 类入门:计数器

```
// C 的做法:struct Counter { int cnt; }; + 全局函数 counter_inc(&c)
// C++ 的做法:数据与操作放进同一个 class,成员函数直接操作自己的数据
class Counter {
    int cnt_ = 0;                // 成员变量(默认私有,外部不可见)
public:
    void inc() { ++cnt_; }        // 成员函数:修改自己的状态
    void reset() { cnt_ = 0; }
    int get() const { return cnt_; } // const 成员函数:承诺不修改对象
};
// 使用:Counter c; c.inc(); c.inc(); cout << c.get(); // 输出 2
```

1.4.5 编译运行与完整小程序

本书 C++ 代码统一按 C++17 标准编译: `g++ -std=c++17 hello.cpp -o hello && ./hello` (MSVC: `cl /EHsc /std:c++17 hello.cpp`)。建议加 `-Wall -Wextra` 打开全部警告——警告是免费的调试器。

示例 1-5 完整小程序:学生成绩统计(综合本节全部知识点)

```
// hello.cpp — 第一个完整 C++ 程序:学生成绩统计
// 输入:学生姓名与分数;输出:平均分与最高分
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Student {                                // 类:姓名 + 分数
public:
    string name;
    int score;
    Student(string n, int s) : name(n), score(s) {} // 构造函数
};

int main() {
    int n;
    cout << "学生人数: ";
    cin >> n;
    vector<Student> stu;                        // 动态数组:存 Student 对象
    for (int i = 0; i < n; ++i) {
        string name;
        int score;
        cin >> name >> score;                  // 读入一行中的两个值
        stu.push_back(Student(name, score));
    }
    int sum = 0, best = 0;
    for (const Student& s : stu) {              // 范围 for:引用避免拷贝
        sum += s.score;
        if (s.score > best) best = s.score;
    }
    cout << "平均分: " << sum / n << ",最高分: " << best << endl;
    return 0;
}
// 运行示例:
// 学生人数: 3,输入:张三 90、李四 85、王五 95
// 平均分: 90,最高分: 95
```

§1.5 完整示例:斐波那契

1.5.1 朴素递归:指数级

斐波那契数列: $F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)$ 。照定义直接写递归——“正确但低效”:

示例 1-6 朴素递归斐波那契, $O(2^n)$

```
// F(0)=0, F(1)=1, F(n) = F(n-1) + F(n-2)
// 朴素递归:F(k) 被重复计算无数遍(违反合成效益法则)
long long fibNaive(int n) {
    if (n <= 1) return n;                        // 基准情形
    return fibNaive(n - 1) + fibNaive(n - 2);    // 不断推进,但一裂为二
}

// 调用次数 =  $2 \cdot F(n+1) - 1$  ( $F(40)$  约 3.3 亿次),  $n$  每加 10 约乘 120
// 递推式  $T(n) = T(n-1) + T(n-2) + O(1) \rightarrow O(2^n)$ 
```

⚠ 误区 3:"递归只是慢一点"——是数量级爆炸

同一输入,朴素版 $F(40)$ 约 3.3 亿次调用(本机约 0.1 秒,慢的机器数秒),记忆化版(下节)约 41 次调用——相差约八百万倍,不是"几倍"。指数级增长意味着:输入每加一点,时间乘一个常数—— n 从 40 到 50,调用数再乘约 120。另外, $F(47)$ 就超过 32 位 int 范围,示例统一用 `long long`。

1.5.2 记忆化:线性

观察调用树: $F(5)$ 要算 $F(4)$ 、 $F(3)$, $F(4)$ 又要算 $F(3)$ 、 $F(2)$ —— $F(3)$ 被重复计算,更小的 $F(k)$ 被重复更多次。解决办法是**记忆化**(memoization):第一次算出 $F(k)$ 就存进表格,以后直接查表。每个 $F(k)$ 只算一次,共 $n+1$ 个子问题,总计 $O(n)$ 。

示例 1-7 记忆化斐波那契, $O(n)$

```
// 记忆化:算过的 F(k) 存进数组,下次直接取
// 每个 F(k) 只算一次,调用次数从指数级降到 O(n)
#include <vector>
using namespace std;

long long fibMemo(int n, vector<long long>& memo) {
    if (memo[n] != -1) return memo[n];    // 已经算过:直接返回(查表)
    if (n <= 1) return n;                // 基准情形
    memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    return memo[n];                      // 记住结果,避免重复劳动
}

long long fibMemoWrapper(int n) {
    vector<long long> memo(n + 1, -1);    // 全部初始化为"未算过"
    return fibMemo(n, memo);
}

// fibMemoWrapper(90) 立刻返回
```

1.5.3 手写 vs 标准库

记忆化是"手写优先"的典型:C++ 标准库没有内置记忆化/缓存(不像 Python 的 `functools.lru_cache`),工程中要么手写数组/散列表,要么用第三方库。作为"手写 vs 标准库"的对照,看个更简单的例子——整数幂:

示例 1-8 手写整数幂 vs `std::pow`

```
// 手写:循环 n 次相乘,  $O(n)$  (更快的方法"快速幂"见第2章)
long long powHand(int a, int n) {
    long long r = 1;
    for (int i = 0; i < n; ++i) r *= a;
    return r;
}

// 标准库: std::pow 处理浮点 double, 参数可以是任意实数
#include <cmath>

double x = std::pow(2.0, 10);            // 1024.0(浮点)
// 对照:手写版返回整数,只支持自然数次幂; std::pow 返回 double
// 结论:整数次幂用自写循环;涉及取模必须自写
// 记忆化同理:无标准库,手写数组是标准做法
```

调试技巧:斐波那契的三种对照验证

写记忆化版最容易犯"数组越界"与"忘初始化"两个错。三个验证技巧:① 先跑小 $n(n \leq 10)$,与手算值对照;② 朴素版与记忆化版互相对拍($n \leq 30$ 内朴素版也很快);③ 记忆化数组大小必须是 $n+1$ (存下标 $0..n$),初始化成"未算过"的哨兵值(如 -1)——斐波那契值恒非负, -1 不会与真实结果冲突。

1.5.4 迭代版:空间也省下来

还能更省:只保留前两个值滚动前进,时间 $O(n)$ 、空间 $O(1)$: `long long a = 0, b = 1;` 循环 `t = a + b; a = b; b = t;`。三版对比:朴素 $O(2^n)$ 时间、记忆化 $O(n)$ 时间 $O(n)$ 空间、迭代 $O(n)$ 时间 $O(1)$ 空间。记忆化解决"重复计算",迭代再省掉栈空间。

§1.6 递归的工程注意

1.6.1 调用栈与栈帧

§1.3.2 说过:每次函数调用压入一个**栈帧**(返回地址、参数、局部变量)。一个栈帧通常几十到几百字节,递归深度 10^5 时就要数 MB 内存。

1.6.2 栈溢出与段错误

C++ 程序的调用栈大小由系统/编译器决定,默认约 8MB。递归深度超过栈容量时,程序写入栈以外的地址,操作系统终止进程——表现为**段错误(Segmentation fault)**:没有任何报错信息,进程直接消失,也无法 try/catch。

示例 1-9 深层递归:段错误现场(不要直接跑)

```
// 递归求和:逻辑完全正确,但 n = 1000000 时崩溃
#include <iostream>
using namespace std;
long long sumRec(int n) {
    if (n == 0) return 0;           // 基准情形
    return n + sumRec(n - 1);       // 需要 n 层栈帧同时存活
}
int main() {
    // 深度 100 万 × 每帧几十字节 > 默认栈约 8MB
    cout << sumRec(1000000) << endl; // 段错误:栈空间耗尽,进程崩溃
    return 0;
}
```

误区 4:栈溢出无法预测

栈溢出在哪个深度爆发取决于每帧大小与栈大小:局部变量多(大数组、多参数)的帧更大,同样深度先爆。更隐蔽的是**测试规模太小测不出问题**: $n = 10^4$ 跑得好好的,生产环境 $n = 10^6$ 上线即崩。规则:线性递归的深度就是 n ,凡 n 可能超过约 10^5 的,一律不写线性递归。

1.6.3 如何避免深层递归

示例 1-10 改写为迭代:同样正确,百万规模轻松完成

```
// 用循环改写:不再占用调用栈,百万规模轻松完成
long long sumIter(int n) {
    long long s = 0;
    for (int i = 1; i <= n; ++i) s += i;
    return s;
}
// 识别"可改迭代"的标志:递归调用在函数末尾且结果直接返回
// (尾递归形态;编译器也可能自动优化,但不要依赖)
// 第3章会学到"显式栈"这一通用替代方案
```

避免深层递归的三招:① **循环改写**——尾递归形态(递归调用在函数末尾)最容易改写,空间从 $O(n)$ 变 $O(1)$;② **显式栈**——用自己维护的栈模拟调用栈(第3章),不受系统栈限制;③ **控制深度**——设计"减半"式递归(分治,第2章),深度只有 $\log_2 n$, 10^6 规模也不到 20 层。

§1.7 全书路线图

先看完整地图再出发。全书 12 章,一句话导航:

- **第2章 算法分析**:大 O 记号、增长阶排序、运行时间计算规则、最大子序列和四种算法
- **第3章 表、栈和队列**:三种最基础 ADT,数组与链式两种实现,栈的三个经典应用
- **第4章 树**:遍历、表达式树、二叉查找树 BST、AVL 平衡
- **第5章 散列**:散列函数、分离链接与开放定址两种冲突处理
- **第6章 优先队列(堆)**:二叉堆、堆排序、d-堆与左式堆
- **第7章 排序**:插入/希尔/堆/归并/快排/桶排,稳定性与复杂度全景
- **第8章 不相交集**:等价关系、按秩合并与路径压缩,接近 $O(1)$ 的并查集
- **第9章 图论算法**:DFS/BFS、拓扑排序、最短路径、最小生成树
- **第10章 算法设计技巧**:贪心、分治、动态规划、回溯、随机化
- **第11章 摊还分析**:聚集、会计、势能三种方法,解释"平均 $O(1)$ "的严格依据
- **第12章 高级数据结构**:伸展树、B 树、跳表等进阶结构

依赖关系:第2章是后续所有复杂度结论的依赖,务必学透。每章结束回看这张图,知道自己走到了哪里。

本章复杂度速查表

操作 / 算法	数据结构 / 算法	平均	最坏
求 a^n	朴素连乘(手写循环)	$O(n)$	$O(n)$
求 a^n	快速幂(第2章)	$O(\log n)$	$O(\log n)$
求 $\log_b X$	换底公式 + 库函数 $\log$	$O(1)$	$O(1)$
级数求和	算术级数(公式直接算)	$O(1)$	$O(1)$
级数求和	几何级数(公式直接算)	$O(1)$	$O(1)$
级数求和	调和级数(逐项累加)	$O(n)$	$O(n)$
整数取模	C/C++(商向 0 截断)	$O(1)$	$O(1)$
整数取模	Python(商向下取整)	$O(1)$	$O(1)$
阶乘 $n!$	递归(线性递归)	$O(n)$	$O(n)$
阶乘 $n!$	迭代循环	$O(n)$	$O(n)$
斐波那契 $F(n)$	朴素递归	$O(2^n)$	$O(2^n)$
斐波那契 $F(n)$	记忆化递归	$O(n)$	$O(n)$
斐波那契 $F(n)$	迭代滚动(两个变量)	$O(n)$	$O(n)$
递归调用	递推式 $T(n) = T(n-1) + O(1)$	$O(n)$	$O(n)$
递归调用	递推式 $T(n) = T(n-1) + T(n-2) + O(1)$	$O(2^n)$	$O(2^n)$
调用栈空间	C++ 递归深度 n (默认栈约 8MB)	$O(n)$ 空间	深度过大即段错误
递归深度上限	Python 默认 1000 层	1000 层	可调大,但过大有崩溃风险

最小必会清单

- 能默写指数四条规则与里程碑数值(2^{10} 、 2^{20} 、 2^{30})
- 能说出对数定义、换底公式与 $\log/\log_2/\ln$ 约定,解释"复杂度中底数不重要"
- 能默写算术、几何、调和三个级数公式并说出各自量级
- 能说出 C/C++ 与 Python 负数取模的区别($C++:-7\%3=-1$; $Python:-7\%3=2$),并写出"非负余数"修正惯用法
- 能说出"程序 = 数据结构 + 算法"并举出数据结构影响算法难度的例子
- 能默写递归四法则,并指出朴素斐波那契违反合成效益法则
- 能独立写出阶乘的递归版与迭代版,说出递推式 $T(n) = T(n-1) + O(1)$ 的解是 $O(n)$
- 能独立写出朴素与记忆化斐波那契,并解释复杂度为何是 $O(2^n)$ 与 $O(n)$
- 能用 `g++ -std=c++17` 编译运行一个含 `cin/cout`、`string`、`vector`、`class` 的完整程序
- 能解释栈帧、调用栈与深层递归为何段错误;说出三种避免深层递归的手段
- 能说出本书 12 章的大致内容与依赖关系

自测题

Q 1. 计算: 2^{10} 是多少? $\log_2 1024$ 是多少? $\sum_{i=1}^{100} i$ 是多少?

✓ 参考答案

$2^{10} = 1024$; $\log_2 1024 = 10$ ($2^{10} = 1024$); $\sum_{i=1}^{100} i = 100 \times 101 / 2 = 5050$ 。分别对应指数、对数、算术级数三公式。

Q 2. C++ 中 $-7 \% 3$ 与 $7 \% -3$ 分别等于多少?为什么?与 Python 有何不同?

✓ 参考答案

$-7 \% 3 = -1$, $7 \% -3 = 1$; C/C++ 除法向 0 截断, $-7/3$ 取商 -2, 余数 $-7 - (-2) \times 3 = -1$; $7/-3$ 取商 -2, 余数 $7 - (-2) \times (-3) = 1$ 。Python 向下取整, 结果分别为 2 与 -2; 同号一致, 异号不同。

Q 3. 递归四法则是什么?朴素递归斐波那契违反了哪一条?

✓ 参考答案

① 基准情形: 存在不需递归的直接答案; ② 不断推进: 每次调用都朝基准情形靠近; ③ 设计法则: 假设所有递归调用都正确, 只组装当前层; ④ 合成效益法则: 不在调用间重复求解同一实例。朴素斐波那契违反第④条, 复杂度爆炸到 $O(2^n)$ 。

Q 4. 递推式 $T(n) = T(n-1) + O(1)$ 的解是什么?给出直觉推导。

✓ 参考答案

解为 $O(n)$ 。展开: $T(n) = T(n-1) + c = T(n-2) + 2c = \dots = T(0) + n \cdot c$, 共 n 步, 每步常数代价。阶乘与递归求和都是这个递推式。

Q 5. 朴素递归斐波那契计算 $F(40)$ 大约需要多少次递归调用?记忆化后呢?

✓ 参考答案

朴素版调用次数 $= 2 \times F(41) - 1 \approx 3.3$ 亿次 (本机约 0.1 秒); 记忆化后每个 $F(k)$ 只算一次, 约 41 次调用, 瞬间完成。这就是合成效益法则的意义。

Q 6. 为什么递归 10 万层的 C++ 程序会段错误?栈帧在其中扮演什么角色?

✓ 参考答案

每次调用压入一个栈帧 (返回地址、参数、局部变量), 深度 10^5 层 $\times$ 每帧几十字节 $\approx$ 数 MB, 超过默认调用栈 (约 8MB) 即溢出, 写入非法地址被操作系统终止, 表现为无报错的段错误。规避: 循环改写、显式栈、log 深度的分治递归。

章末实验题:递归综合实验:汉诺塔与全排列

实验 1:汉诺塔与全排列(递归综合)

实验目标:用递归实现汉诺塔与全排列,验证 $2^n - 1$ 与 $n!$ 两个计数结论,并用真实计时感受指数级增长。

实验要求:

- 任务 1:汉诺塔——递归打印移动步骤、统计总步数,验证 $n = 1..10$ 步数恒等于 $2^n - 1$ (知识点:\$1.3 递归四法则)
- 任务 2:递归生成全排列——交换 + 回溯,验证 $n = 3、4$ 的排列总数为 $3! = 6、4! = 24$ (知识点:\$1.3 树形递归、回溯思想)
- 任务 3:对比 n 增大时朴素递归斐波那契的时间增长——记录 $n = 32/35/38$ 的耗时,验证调用数按 $\varphi^3 \approx 4.2$ 倍增长(知识点:\$1.5 指数级复杂度)

实验环境:C++17,g++ 或 MSVC。编译运行: `g++ -std=c++17 lab1.cpp -o lab1 && ./lab1` (MSVC: `cl /EHsc /std:c++17 lab1.cpp`)。

第一步 写 hanoi:基准情形 $n == 1$ 直接搬;否则三步——上面 $n-1$ 个搬到中转柱、搬最大盘、再搬 $n-1$ 个。注意三次调用参数轮换顺序

第二步 用一个全局计数器统计步数;打印开关只对 $n = 3$ 打开, $n = 1..10$ 只输出"实际 vs 理论 $2^n - 1$ "对照行

第三步 permute:固定前 k 位;把 $a[k..]$ 中每个元素换到第 k 位;递归排列剩余部分;再换回还原现场(回溯三件套);基准情形 $k == n$ 时输出当前排列

第四步 时用 `chrono::steady_clock`,对 $n = 32/35/38$ 各测一次,打印耗时并观察"每加 3 乘几"的比值

✅ 参考答案(完整可运行程序,约 80 行,分两段)

lab1.cpp(第一段:任务 1 汉诺塔与任务 2 全排列)

```
// lab1.cpp — 递归综合实验:汉诺塔与全排列(第一段:任务1与任务2)
#include <iostream>
#include <vector>
#include <chrono>
using namespace std;

long long moveCnt = 0; // 汉诺塔移动总步数

// 任务 1:汉诺塔。printMove 为 true 时打印每步,否则只计数
void hanoi(int n, char from, char via, char to, bool printMove) {
    if (n == 1) { // 基准情形:只剩一个盘子,直接搬
        if (printMove) cout << " " << from << " -> " << to << endl;
        ++moveCnt;
        return;
    }
    hanoi(n - 1, from, to, via, printMove); // 1. 上面 n-1 个搬到中转柱
    if (printMove) cout << " " << from << " -> " << to << endl;
    ++moveCnt; // 2. 最大盘子搬到目标柱
    hanoi(n - 1, via, from, to, printMove); // 3. n-1 个从中转柱搬到目标
}

// 任务 2:递归生成全排列(回溯)。固定前 k 位,把 a[k..] 逐个换到 a[k]
void permute(vector<int>& a, int k, long long& cnt) {
    if (k == (int)a.size()) { // 基准情形:最后一位也固定了
        for (size_t i = 0; i < a.size(); ++i)
            cout << a[i] << (i + 1 == a.size() ? "\n" : " ");
        ++cnt;
        return;
    }
    for (int i = k; i < (int)a.size(); ++i) {
        swap(a[k], a[i]); // 把 a[i] 换到第 k 位
        permute(a, k + 1, cnt); // 递归排列剩余部分
        swap(a[k], a[i]); // 回溯:换回来,还原现场
    }
}
```

lab1.cpp(第二段:任务 3 计时与主程序)

```

// 任务 3:朴素递归斐波那契(用于计时)
long long fibNaive(int n) {
    if (n <= 1) return n;
    return fibNaive(n - 1) + fibNaive(n - 2);
}

int main() {
    cout << "==== 任务 1:汉诺塔(n = 3)====\n";
    hanoi(3, 'A', 'B', 'C', true);
    cout << "移动步数: " << moveCnt << "(理论 2^3 - 1 = 7)\n";
    moveCnt = 0; // 清零,再进入批量验证
    for (int n = 1; n <= 10; ++n) { // 验证 2^n - 1
        hanoi(n, 'A', 'B', 'C', false);
        long long expect = (1LL << n) - 1;
        cout << "n = " << n << ": 实际 " << moveCnt
            << ",理论 " << expect
            << (moveCnt == expect ? " 一致" : " 不一致!") << endl;
        moveCnt = 0;
    }

    cout << "\n==== 任务 2:全排列(n = 3)====\n";
    vector<int> a = {1, 2, 3};
    long long permCnt = 0;
    permute(a, 0, permCnt);
    cout << "排列总数: " << permCnt << "(理论 3! = 6)\n";
    permCnt = 0;
    vector<int> b = {1, 2, 3, 4}; // 再验证 4! = 24
    permute(b, 0, permCnt);
    cout << "n = 4 排列总数: " << permCnt << "(理论 4! = 24)\n";

    cout << "\n==== 任务 3:朴素斐波那契计时 ==== \n";
    for (int n : {32, 35, 38}) { // n 每次 +3,调用数约 x4.2
        auto t0 = chrono::steady_clock::now();
        long long f = fibNaive(n);
        double ms = chrono::duration<double, milli>
            (chrono::steady_clock::now() - t0).count();
        cout << "fib(" << n << ") = " << f << ",耗时 " << ms << " ms\n";
    }
    cout << "观察: n 每加 3,耗时约乘 4~5(调用数 2*F(n+1) 的比值约 phi^3 = 4.2)\n";
    return 0;
}

```

运行命令: g++ -std=c++17 lab1.cpp -o lab1 && ./lab1 。示例输出(第一段,n = 3 的移动过程与 1..10 验证):

===== 任务 1:汉诺塔(n = 3)=====

A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C

移动步数: 7(理论 $2^3 - 1 = 7$)

n = 1: 实际 1,理论 1 一致
n = 2: 实际 3,理论 3 一致
n = 3: 实际 7,理论 7 一致
n = 4: 实际 15,理论 15 一致
n = 5: 实际 31,理论 31 一致
n = 6: 实际 63,理论 63 一致
n = 7: 实际 127,理论 127 一致
n = 8: 实际 255,理论 255 一致
n = 9: 实际 511,理论 511 一致
n = 10: 实际 1023,理论 1023 一致

示例输出(第二段,任务 2 与任务 3;任务 3 的耗时因机器而异,比值规律一致):

===== 任务 2:全排列(n = 3)=====

1 2 3
1 3 2
2 1 3
2 3 1
3 2 1
3 1 2

排列总数: 6(理论 $3! = 6$)

n = 4 排列总数: 24(理论 $4! = 24$)(n = 4 的 24 行清单略)

===== 任务 3:朴素斐波那契计时 =====

fib(32) = 2178309,耗时 1.67 ms

fib(35) = 9227465,耗时 6.61 ms

fib(38) = 39088169,耗时 30.7 ms

观察: n 每加 3,耗时约乘 4~5(调用数 $2 * F(n+1)$ 的比值约 $\phi^3 = 4.2$)

设计说明:① 三个任务共用"基准情形 + 不断推进"的递归骨架:汉诺塔基准是 $n == 1$,全排列是"最后一位也固定",斐波那契是 $n \leq 1$ ——同一思想的三次演练;② 汉诺塔步数 $2^n - 1$ 正是第2章递推式 $T(n) = 2T(n-1) + O(1)$ 的原型;③ 全排列的"交换 → 递归 → 换回"是回溯法(第10章)的最小模板;④ 任务 3 用真实计时把指数级增长变成肉眼可见(实测 n 每加 3 耗时约乘 4~5,调用数比值 $\phi^3 \approx 4.2$);⑤ 打印与计数分离(printMove 开关)让"演示"与"批量验证"互不干扰。

下一章预告:第2章 算法分析——为大 O 记号建立严格定义,学会对循环与递归做复杂度计算,并把最大子序列和从 $O(n^3)$ 优化到 $O(n)$ 。本章的递推式、对数与指数性质、级数公式,正是第2章推导的数学地基。